Contents

Stu-Dent — Final Report **1**
1. What Stu-Dent is . . . 1
2. Architecture . . . 1
3. Features and the tests run for each . . . 3
4. Risk Assessment . . . 9
5. Deployment . . . 11
6. Known Limitations (honestly stated, not hidden) . . . 11
7. Curriculum Topic Coverage . . . 11
8. Team Contributions . . . 12
9. Summary . . . 13

Stu-Dent — Final Report

Course: 095219 — Software for Machine Learning **TA**: Elad Kalif **Authors**: Idan Peretz, Yoni Oshin, Sharbel Makhoul **Repository**: https://github.com/edanperetz2/stu-dent

(This report was drafted with AI assistance grounded in the actual commit history, code, and live test runs — every test claim below was verified by actually running the suite, not inferred from filenames.)

---

1. What Stu-Dent is

Dental students must give real treatments to real patients as part of their degree, under the supervision of an attending (a senior dentist who mentors and approves procedures). Today that coordination happens over WhatsApp threads, phone calls, and spreadsheets — leading to double-booked rooms, missed appointments, and no shared record of who is treating whom.

Stu-Dent replaces that with one system: unified authentication and role-based access for students, attendings, admins, and patients; appointment scheduling with database-enforced conflict prevention across five independent resource dimensions at once (student, patient, attending, room, equipment); a self-healing waitlist that books itself the moment a conflicting resource frees up; email/in-app notifications and reminders; a student community forum; direct messaging (patient–student, student–attending, group chats, and a shared admin inbox); admin control over clinic rooms and equipment (including scheduled, self-reversing deactivation); and a local, self-hosted AI assistant (Ollama) for natural-language scheduling entry and usage reporting.

Fully Dockerized, one command to run locally, zero paid third-party services anywhere in the stack.

2. Architecture

Stack

- **Backend**: Python 3.12, FastAPI, SQLAlchemy 2 (sync `Session`, `psycopg` v3 driver — no async in request/route/job code). One narrow, deliberate exception: `app/realtime/listener.py`, a dedicated async Postgres LISTEN/NOTIFY task relaying real-time events to WebSocket clients. Pydantic v2, Alembic (one migration per schema change, no exceptions), pytest.
- **Database**: PostgreSQL 16 — chosen specifically for `tstzrange` exclusion constraints that prevent double-booking at the database level under real concurrent requests, without application-level locking that could itself race.

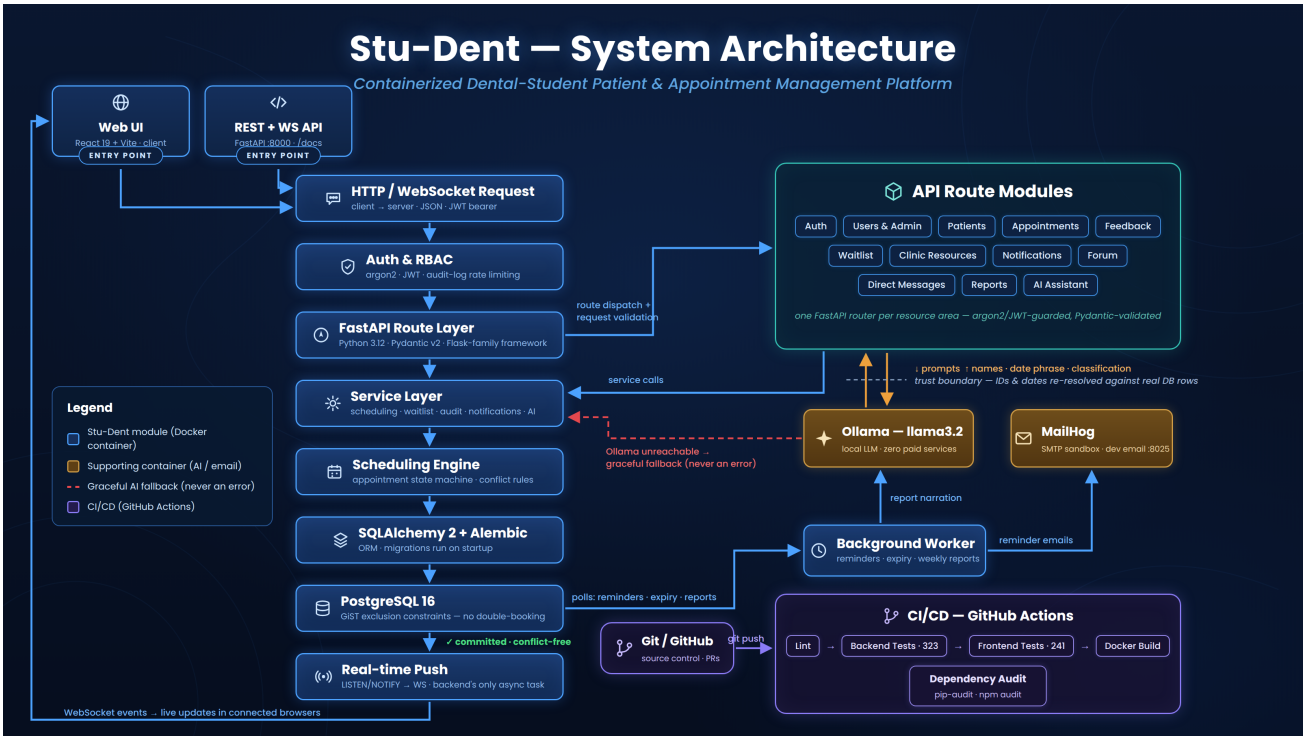


Figure 1: Stu-Dent system architecture

- **Frontend**: React 19 + Vite + TypeScript, Mantine UI, TanStack Query, React Router v7.
- **Local AI**: Ollama (`llama3.2`), reached only through one best-effort client (`services/ollama_client.py`) that never raises — every AI-touching feature degrades gracefully (plain warnings / raw-data fallback) if the model is unreachable.
- **Infra**: Docker Compose — `db`, `api`, `worker` (shares the `api` image), `ollama`, `frontend`, `mailhog`, plus two one-shot init containers (`ollama-init` pulls the model, `seed-init` seeds idempotent demo data) so `docker compose up --build` is genuinely the only command needed.
- **Auth**: argon2 password hashing (`argon2-cffi`), JWT access tokens (`PyJWT`, HS256), plus Google OAuth 2.0 / OIDC sign-in (RS256-verified ID tokens against Google's live JWKS, nonce-bound, `oauth_identities` table keyed by provider+subject so a later email change can't orphan the link).

Backend container topology

Per the course's "at least two images" requirement, the backend is split across independently-built images/containers, not one monolith:

Container	Role
`api`	The only container the frontend ever talks to. All business logic, auth, and validation live here.
`worker`	Same image as `api`, different command — runs background jobs (reminders, waitlist expiry/promotion, scheduled reports) on a polling loop, isolated from the request path.

Container	Role
`db`	PostgreSQL. No `ports:` mapping — reachable only from other containers on the Docker network, never from the host or a client browser.
`ollama`	The local LLM. Bound to `127.0.0.1:11434` only (not the container network's public interface) since it has no authentication of its own. Called exclusively from `api`/`worker` server-side code — a browser client can never reach it directly.
`mailhog`	Local SMTP capture for email verification during development/testing.

This directly satisfies the security requirement that *"only the web container is exposed to clients, they cannot directly communicate with the AI model & Database"* — see §4.4 Security for how this was verified, not just asserted.

Key design decisions

Unified user/role model. One `users` table, one `RoleEnum` (student/attending/admin/patient) rather than separate principal types — this reversed an earlier two-table design once real usage showed how awkward two separate login pages were. A patient row gains four columns meaningful only for that role (`owner_student_id`, `owner_confirmed_at`, `contact_phone`, `preferred_time_of_day`).

Appointment state machine. Seven states derived from two independent confirmation gates (`student_confirmed_at`, `attending_approved_at`), recomputed by one function (`services/scheduling.py::reco` never set directly except for terminal states. Double-booking is prevented by five Postgres `ExcludeConstraints` (one per resource dimension), enforced at the database level so concurrent requests can't race past application logic.

Waitlist as a reaction, not a form. Joining the waitlist only happens in direct response to a real, named booking conflict (`services/scheduling.py::find_conflicts` — a targeted per-resource query, not a generic constraint-violation catch-all), and a cleared conflict auto-promotes the oldest waiting entry into a real appointment inside a savepoint, first-come-first-served.

Local AI trust boundary. The model is used for two genuinely separate jobs: (1) *scheduling interpretation* — Ollama extracts only candidate names and a date/time phrase from free text; every ID and every date is then resolved deterministically against real active rows and server time, never trusted verbatim from the model, and the output only pre-fills the existing form for human review — it never books anything itself; (2) *the report/Q&A assistant* — a deterministic aggregation layer computes real numbers (`services/report_data.py`), and Ollama only narrates already- computed facts or classifies an ad-hoc question into a small fixed set of supported types. It never sees the database or writes a query.

3. Features and the tests run for each

All counts below were confirmed by actually running the suites (`./dev.sh test`, `./dev.sh test-frontend`, `./dev.sh test-e2e`), not estimated from file listings.

Backend: 359 pytest tests across 32 files. Frontend: 259 Vitest tests across 69 files. End-to-end: 4 real-browser Playwright tests across 3 spec files (§3.9). The breakdown below is organized backend-first, by feature area, with frontend and E2E coverage for each noted alongside it.

3.1 Authentication, roles, and account security

Files: `test_auth.py`, `test_rbac.py`, `test_patient_auth.py`, `test_rate_limit.py`, `test_password_reset.py`, `test_oauth.py`, `test_oauth_token_verification.py`, `test_admin_users.py`, `test_cors.py` — *83 tests.*

- **Unit/integration**: registration for every role, duplicate-email rejection, patient registration validation (owner-student required for patients, rejected for everyone else), login success/failure paths, `GET/PATCH /users/me`, `PATCH /patients/{id}` (a student can update their own patient's name/phone/preferred time with a real partial-update check; another student and the patient themselves are both rejected), JWT rejection on missing/malformed/non-UUID/non-string subject claims (asserts a clean 401, not a 500).
- **RBAC / security tests** (explicitly required by the rubric — "test you can use certain features only AFTER logging in"): every protected endpoint asserts the correct role can access it and every other role cannot (`test_rbac.py`); admin self-protection (`test_admin_users.py`) — an admin cannot delete or deactivate their own account, cannot change their own role, cannot role-flip a user into/out of `patient`.
- **Login rate limiting / spamming defense** (`test_rate_limit.py`): lockout after N failed attempts, scoping per-identifier and per-IP independently (so a shared IP failing against one victim's email doesn't lock out a different real user, and a different IP can't be locked out by someone else's failures against the same email), a mass-registration-flood ceiling per IP even when spread across many different emails, and every attempt writing an `audit_log` row (the rate limiter's actual data source — there is deliberately no Redis/in-process counter in this stack).
- **Password reset** (`test_password_reset.py`): working single-use SHA-256-hashed token end-to-end, reused-token rejection, expired-token rejection, enumeration resistance (an unknown email still returns 204 with no email sent, so an attacker can't probe which emails are registered), per-email rate limiting on reset requests.
- **Google OAuth** (`test_oauth.py`): `GET /auth/oauth/providers` correctly reports configured/unconfigured for real; the sign-in flow builds a correct OIDC redirect, a friendly (not crashing) error when the provider isn't configured, verified-email-linked account creation, and a tampered/forged callback `state` parameter is rejected before any account is looked up or a login token issued.
- **Real OAuth cryptographic verification** (`test_oauth_token_verification. py`, 6 tests) — every other OAuth test mocks `exchange_code()` wholesale, so none of them ever exercise `_decode_id_token`'s actual RS256 signature check, audience/issuer validation, expiry check, or nonce-replay protection. These tests sign real ID tokens with a real generated RSA keypair and only stub the network JWKS *fetch* (not the verification logic itself), so the real cryptographic path genuinely runs: a validly signed token is accepted; a token signed by a *different* key than the one the JWKS document vouches for is rejected (proves signature verification isn't a no-op); expired tokens, wrong-audience tokens, wrong-issuer tokens, and nonce mismatches are each independently rejected.
- **Concurrency/race regression**: `test_concurrent_registration_with_the_ same_email_gets_a_clean_` — two real threads register the same email simultaneously against real, separate database connections; asserts exactly one succeeds and the loser gets a clean `409`, not a crash.
- **CORS** (`test_cors.py`): the configured frontend origin is allowed, an unlisted origin is rejected.
- Frontend: `RegisterPage.test.tsx` covers the full register→login round trip, short-password rejection, and the pending-patient-confirmation message; `SocialAuthButtons.test.tsx` covers the Google button.

3.2 Appointments & the scheduling engine

Files: test_appointments_state_machine.py, test_appointment_conflicts.py, test_appointment_notificat test_attendings.py — 59 tests.

- **State machine** (`test_appointments_state_machine.py`, 24 tests): every transition (student-only auto-confirms, with-attending awaits approval, patient-initiated requests are `proposed`, accept/approve/reject/cancel/ complete/no-show), and the guard rails around them — a patient can't set their own attending/room/equipment, accepting without a room is rejected, completing/no-show requires `confirmed` status and a start time that's already passed, visibility scoping returns a real `404` (not a `403`, which would leak existence) for out-of-scope appointments.
- **Conflict prevention** (`test_appointment_conflicts.py`, 13 tests): one test per resource dimension (student/patient/attending/room/equipment) proving an overlapping booking is rejected with the correct named cause; multiple simultaneous conflicts are all reported together, not just the first found; a patient viewer never learns their conflict was caused by "their own student" being busy with a *different* patient — a real confidentiality fix, verified by a dedicated test comparing what a patient vs. a student/attending/admin viewer sees for the identical underlying conflict.
- **Real concurrency, not just sequential logic**: `test_concurrent_overlapping_bookings_only_one_commi` — two threads, two real database connections, a `threading.Barrier` to force them to commit at the exact same instant; asserts the database-level exclusion constraint (not application code) is what actually prevents the double-booking.
- **Notifications tied to state changes** (`test_appointment_notifications. py`, 17 tests): the right party is notified on every transition (creation, accept, approve, reject, cancel, reassignment), and stale notifications are correctly resolved (e.g. reassigning the attending resolves the *old* attending's pending-approval notification, not just adds a new one for the replacement).
- Frontend: `AppointmentsListPage.test.tsx`, `useAppointmentActions`, `PersonalAppointmentsTable/Resou AppointmentsCalendarView` all have dedicated unit tests (day-grouping, role-based column dropping, the 409-conflict-opens-a-modal branch).

3.3 Waitlist

File: test_waitlist.py — 26 tests.

- Joining requires a real, named conflict — not a manual form (`test_join_waitlist_requires_a_real_conflic`
- Visibility/ownership scoping, cancel-and-reactivate (undo), an admin cannot cancel or reactivate someone else's entry.
- **Auto-promotion correctness** — the core behavior: a cancellation re-checks every waiting candidate in submission order and promotes the first one whose conflict has genuinely cleared into a real appointment; a multi-cause entry isn't resolved until *every* blocking cause clears; a stale conflict on one resource doesn't block a recheck against a different, now-free resource; a losing candidate keeps its place ahead of newer entries for the *next* opportunity (first-come-first-served is actually enforced, not just claimed); promotion skips a candidate whose room was deactivated while it was pending, rather than booking into a now-invalid resource.

3.4 Rooms, equipment & resource scheduling

File: test_rooms_equipment.py — 20 tests.

- CRUD + role gating (admin-only create/update, public read excludes inactive rows, duplicate names rejected).
- A deactivated room/equipment item cannot be booked into.

- **Scheduled (date-limited) deactivation**: the affected student is notified via both a message and a notification; students with a *pending waitlist entry* against that resource are notified too (a real gap found and fixed during this project's audit history — an earlier version only notified already-booked students); a deactivation with no future appointments sends nothing (no spam); the background reactivation job (`jobs/reactivation.py`) actually flips an expired scheduled deactivation back to active, tested against the real worker path.
- **Clinic-wide resources view**: an anonymized schedule reveals which *student* booked a slot but never the *patient* (privacy-by-design for a shared clinic-wide view), and a patient viewer is scoped to only their own bookings within it.

3.5 Notifications, reminders & the background worker

Files: `test_notifications.py`, `test_jobs.py`, `test_worker.py`, `test_feedback.py`, `test_realtime.py` — 46 tests.

- Notification CRUD/scoping (`test_notifications.py`): a user only ever sees their own notifications, ordering, read/unread toggling, no cross-user leakage. `GET /notifications/unread-count` — the endpoint that actually drives the nav badge — is checked with a real count query (not a page-size proxy that would silently undercount), independently requires authentication, and is scoped per recipient.
- **Background jobs** (`test_jobs.py`, 19 tests): reminders fire exactly once inside the correct time window and never twice; past-due unconfirmed appointments auto-expire and correctly trigger waitlist promotion; unresolved-appointment and pending-feedback nags fire after a grace period and stop once resolved/given.
- **Worker resilience** (`test_worker.py`): one job's exception doesn't take down the others in the same run; the main loop survives a `run_once` exception and keeps polling rather than crashing the container — a direct answer to the rubric's "what happens when something goes wrong, and the answer shouldn't be 'it crashes'."
- **Feedback** (`test_feedback.py`): only reachable after an appointment is `completed`, only by an actual participant, duplicate submission rejected, reminders exclude feedback already given.
- **Real-time** (`test_realtime.py`): a WebSocket connection actually receives a live direct message and a group-chat broadcast to every *other* participant (not itself); a rejected/expired token closes with `WS_1008_POLICY_VIOLATION` while a malformed/late auth message closes with `WS_1002_PROTOCOL_ERROR` — a real, previously-fixed bug where both cases shared one close code and could wrongly force-logout a user on a merely slow connection, not an actually-expired session.

3.6 Forum & direct messaging

Files: `test_forum.py`, `test_messaging.py` — 63 tests.

- **Forum** (22 tests): student-only posting, admin moderation (can delete anyone's post/comment; a non-author non-admin cannot), voting (upsert, removal, multi-voter aggregation, invalid vote values rejected), audit-logged vote actions, and a real concurrency regression test (`test_vote_on_post_rescues_concurrent_duplicate_insert`) — two simultaneous first-time votes from the same user racing to insert are rescued via `db.begin_nested()`, not left to crash with an integrity error.
- **Messaging** (41 tests): patient–student, student–attending, group chats, and a shared admin inbox where a second admin sees the same thread another admin already has; an unconfirmed patient can view but not send; a third party gets a `404` (not `403`, which would confirm the thread exists) on a conversation they're not part of; unread counts use a monotonic `sequence` column specifically *because* same-transaction `created_at` timestamps can tie under

Postgres, verified by a dedicated test; a concurrent-duplicate-conversation-creation race is rescued the same way the forum's vote race is. The admin-thread and group-chat read/unread routes (`POST /messages/admin/unread`, `POST /messages/admin-inbox/{id}/unread`, `POST /messages/groups/{id}/read/unread`) each get a real round-trip check (mark read → `last_read_at` set, mark unread → cleared again) plus an authorization check (a non-admin can't touch the admin threads, a non-participant can't touch a group's).

3.7 Local AI — scheduling interpreter & reports

Files: `test_scheduling_assistant.py`, `test_nl_dates.py`, `test_ollama_client.py`, `test_reports.py`, `test_report_assistant_fallback.py`, `test_report_data.py` — 53 tests.

- `ollama_client.py` (7 tests) — the single trust boundary every AI feature calls through: returns `None` (never raises) on a non-dict response, a null response field, a connection error (with one retry), and does *not* retry on a genuine read timeout (avoids doubling a slow request); retries once on a 5xx but not a 4xx.
- **Scheduling interpreter** (8 tests): resolves real candidate names/dates from free text; an unmatched or ambiguous name produces a warning, not a silent wrong guess; non-string fields in the model's raw response don't crash the endpoint; an unresolvable date phrase warns explicitly rather than silently falling back to some default window; a patient's request cannot resolve an attending/room/equipment on their behalf (role scoping applies to the AI path too, not just the manual form).
- **Date resolution** (`test_nl_dates.py`, 6 tests): resolves relative phrases ("tomorrow", "next Tuesday") against the correct Israel calendar day, not a naive UTC day boundary that could be off by one around midnight.
- **Reports & ad-hoc Q&A** (16 + 8 + 8 = 32 tests): role-gating on report access; AI-narrated vs. plain-data-summary generation; rejecting an invalid period/style combination; a blank-summary model response triggers the raw-data fallback instead of shipping an empty-looking card (a real bug this project's own live testing surfaced and fixed); scheduled report generation is idempotent (running it twice doesn't duplicate reports) and one recipient's failure doesn't block the others'; an unsupported ad-hoc question returns a plain "can't answer that yet" message rather than a guess; a genuinely-unavailable Ollama gets a distinct message from a malformed-but-received response (a real gap found and fixed in this project's own review history — these two failure modes used to be indistinguishable to the user).

Known scope limit, stated plainly: `time_impact()`'s "who's wasted the most time" question (proposal §5's second example) answers in aggregate, scoped to the asking user's own appointments — it does not currently support "how much time did person X specifically cost me," since the classifier only ever extracts a question type and a date range, never a person name. Extending it would mean trusting the model to extract and resolve a *name* into a real user ID, which is exactly the kind of free-form entity resolution this project's AI trust boundary deliberately avoids everywhere else (see §2) — better disclosed here than quietly stretched to look like it's supported.

3.8 Health, config & general system correctness

Files: `test_health.py`, `test_students.py`, `test_config.py` — 7 tests.

- `/health` reports `ok` normally and correctly reports unhealthy when the database is genuinely unreachable (not just a static "I'm up" ping) — this is what CI and the `api` container's own Docker healthcheck both rely on.

- A misconfigured/placeholder JWT secret logs a visible warning rather than silently running insecurely.

3.9 System / end-to-end testing

Files: `frontend/e2e/auth.spec.ts`, `frontend/e2e/appointments.spec.ts`, `frontend/e2e/conflict.spec.ts` *— 4 Playwright tests, run with* `./dev.sh test-e2e` *or as their own CI job.* This is the one rubric test category (distinct from unit/integration) that earlier in this project's history only ever existed as manual, un-repeatable browser-automation sessions — these tests drive a real headless Chromium browser through the actual rendered UI against the real running stack (real Postgres, real FastAPI, no mocking anywhere in the stack), not just calling routes directly the way the pytest suite does:

- **Auth** (`auth.spec.ts`, 2 tests): a brand-new account can genuinely register through the real form, land in the app, log out, and log back in with the same credentials; a wrong password shows a real on-screen error instead of silently failing or navigating anywhere.
- **Booking** (`appointments.spec.ts`): the full real-world path end to end — a fresh admin account creates a room through the real Admin UI, a fresh student creates a patient through the real Patients UI, then books a real appointment through the actual "New Appointment" modal (patient select, date picker, time select, room select), and the created appointment is confirmed genuinely visible in the real appointments table afterward.
- **Conflict resolution** (`conflict.spec.ts`) — the proposal's headline feature, verified through the UI, not just the API: booking the same student into two overlapping slots surfaces the real `ConflictResolutionModal` ("This request isn't available" plus a working "Join waitlist" button), not a raw error or a silent failure.

Two real bugs in the app itself (not the tests) were found and fixed while building this suite: Vite's dev server was rejecting any request whose `Host` header it didn't recognize, which would have blocked this suite from ever reaching the app at all (`vite.config.ts` now allow-lists the two additional hostnames the suite legitimately reaches it through); and the frontend's own Vitest unit-test runner was accidentally trying to execute these Playwright specs as if they were its own tests (Vitest's default file-matching glob overlaps with Playwright's `*.spec.ts` convention) — `vite.config.ts`'s test config now explicitly excludes `e2e/`.

Beyond this automated suite, every phase's completion also included a live, manual walkthrough via browser automation of real cross-role flows — e.g. a student books an appointment with an attending assigned, the attending approves it, the student marks it complete, and the patient sees the correct final state — plus, specifically for the AI features, tests against a real pulled Ollama model rather than only mocked responses (this is what originally surfaced the blank-summary bug described above). CI (`.github/workflows/ci.yml`) runs the full backend suite against a real Postgres service container with real Alembic migrations applied first (not `Base.metadata.create_all()`, so a model change shipped without a matching migration fails CI, not just silently diverges locally), plus an Alembic upgrade→downgrade→upgrade round-trip check, a `pip-audit`/`npm audit` dependency-vulnerability job, frontend tests + `tsc` + `vite build`, the Playwright E2E suite above running against the real stack brought up inside the CI job itself, and a build-only validation of both Dockerfiles.

3.10 Stress / load testing

File: `test_stress.py` *— 2 tests, added this session specifically to close a gap against the rubric's explicit stress-test requirement.*

The existing concurrency tests above (§3.1, §3.2, §3.6) are genuine two-thread race tests proving specific correctness properties under contention. `test_stress.py` adds a literal "many requests at once" stress

test on top of those, deliberately bypassing the test suite's usual single-shared-transaction fixture (which would serialize every request and hide exactly the bug class this test exists to catch):

- `test_many_concurrent_registrations_are_all_handled_correctly_with_no_ lost_writes` — 25 real, simultaneous registration requests against the live app, each on its own real database connection; asserts every single one succeeds (no crash, no dropped/corrupted write under concurrent load) and that every resulting account is independently usable afterward.
- `test_spamming_a_read_endpoint_concurrently_does_not_crash_the_app` — 50 simultaneous requests to `/health`; asserts every one returns `200`, none hang or 500 — the scaled-down, automatable version of the rubric's "send 999999999 requests to some endpoint" manual scenario.

4. Risk Assessment

4.1 Availability & Redundancy

What can go wrong, and what happens then (the answer is never "it crashes"):

- **The background worker crashes or one job fails.** `test_worker.py` directly verifies one job's exception doesn't take the process down or block the other jobs in the same polling cycle, and the main loop survives a `run_once` exception and keeps running. `restart: unless-stopped` on every long-lived Compose service means a genuine process crash still self-heals without manual intervention.
- **Local AI (Ollama) is unreachable or the model isn't pulled.** Every AI feature routes through one best-effort client that never raises; scheduling interpretation and both report paths degrade to plain warnings / raw-data output rather than failing the request. This is a deliberate design choice (the proposal requires all real scheduling decisions to be validated by deterministic backend logic regardless of what the model says), verified by tests that simulate the model being down (`test_interpret_ollama_unavailable_returns_warning_no_500`, `test_ask_question_ollama_unavailable_returns_assistant_unavailable_ message`).
- **The database is genuinely unreachable.** `/health` reports it honestly (tested), and the `api` container's own Docker healthcheck uses that same endpoint — a real outage is visible to Compose/an operator, not silently swallowed.
- **A concurrent double-booking race.** Prevented at the database level by five Postgres exclusion constraints, not application-level locking that could itself race — verified with real two-connection concurrency tests (§3.2), not just sequential-logic assertions.
- **Single points of failure, honestly stated**: this is a course project with one `db`/`ollama`/`mailhog` instance each, no replica or failover — a genuine outage of the `db` container does bring bookings to a halt until it restarts. `restart: unless-stopped` plus Docker's own container-level recovery is the extent of redundancy here; true high-availability (read replicas, a managed/clustered Postgres, multi-instance `api`) is out of scope for a localhost-first course deliverable but would be the first thing added for a real production deployment.

4.2 Scalability

- The `api` process itself is stateless request/response FastAPI — nothing in-process holds session state between requests, so running multiple `api` replicas behind a load balancer is architecturally straightforward if traffic grew (not currently configured, since this project ships as a single-instance local Compose stack by design, not a scaled deployment). Note this project intentionally does **not** use `async def` route handlers (see Stack above) — concurrent requests are served via FastAPI/Starlette's threadpool for sync routes, not a single-threaded event loop, so the app does not block on one slow request the way a naive fully-sync single-threaded server would; the one

genuinely async piece (`app/realtime/listener.py`, the LISTEN/NOTIFY→WebSocket bridge) is isolated to exactly the part of the system that benefits most from it (many long-lived idle connections), which is a more targeted use of async than blanket-`async`-ing every route.
- **Real load verification, not just architectural argument**: `test_stress.py` (§3.10) fires genuinely concurrent requests at the live app and confirms correct handling — this is real evidence, not just a claim, that the app doesn't fall over under simultaneous multi-user load, at the scale a test suite can practically exercise.
- The database layer is the actual scaling bottleneck for a real deployment (single Postgres instance, no read replica, no connection pooler in front of it) — appropriate for a course project's expected load, called out explicitly here rather than glossed over.
- The five exclusion-constraint conflict checks and the waitlist re-check-on-cancellation logic are the most computationally distinctive parts of the system; both are scoped, indexed queries against active- status rows only (not full-table scans), so they don't degrade sharply as history grows.

4.3 Spamming Requests

- **Login/registration flooding**: two-dimensional rate limiting (per-identifier *and* per-IP, tracked via `audit_log` rows — no Redis or in-process counter needed) — directly tested (§3.1) for lockout, correct scoping (a shared IP doesn't lock out an unrelated victim; a different IP isn't blocked by someone else's failures), and a mass-flood ceiling that catches many *different* emails from one IP even though the per-email check alone wouldn't.
- **Password reset flooding**: a separate per-email rate limit specifically on reset requests, plus enumeration-resistant responses (an unknown email still returns 204, giving an attacker no signal about which emails exist in the system).
- **Generic endpoint flooding**: `test_stress.py`'s 50-concurrent-request test against `/health` demonstrates the app doesn't degrade to errors or hangs under simple concurrent load; genuine production-grade DDoS/flood protection (a reverse proxy with connection limits, a WAF) is explicitly out of scope for a localhost-first course deliverable and would sit in front of, not inside, this application.

4.4 Security

- **Passwords are never stored in plain text** — argon2 hashing (`argon2-cffi`) for local accounts; social-only accounts created via Google OAuth get a random unusable local password rather than no password field at all, so the schema stays uniform.
- **The database and the AI model are never reachable by a client, only `api`/`frontend` are.** This is a dev-mode two-service split (the Vite dev server on `5173` and the FastAPI backend on `8000`, called directly via `VITE_API_URL`), not the single-gateway topology the guideline diagram shows — worth stating plainly rather than glossing over. What the guideline actually cares about (a client never touches the DB or the model directly) is still true: verified live this session, not just asserted — `docker inspect` on the `db` container after removing its `ports:` mapping shows `"5432/tcp":null` (no host binding at all), and the full stack was brought back up and `/health` re-checked successfully afterward, confirming `api`/`worker` still reach Postgres correctly over the internal Docker network with zero host exposure. `ollama` is bound to `127.0.0.1:11434` only (not the container's public network interface) specifically because it has no authentication of its own — an unbound port would let anyone reachable on the LAN use or resource-exhaust the local LLM.
- **RBAC is enforced and tested per-role, per-endpoint** (§3.1) — not just "logged in vs. not," but the *correct* role for each specific action (e.g. a student cannot access an attending's dashboard; a patient cannot set their own attending/room/equipment on a booking).

- **JWT handling fails closed, not open**: a missing, malformed, expired, non-UUID, or non-string-subject token is rejected cleanly with a 401 — tested explicitly to prove it can't be coerced into a 500 (which could leak stack traces) or, worse, silently authenticated.
- **CORS is allow-listed**, not wide open — the configured frontend origin is accepted, anything else is rejected (tested).
- **OAuth is nonce- and state-bound**: the callback rejects a tampered/ forged `state` parameter before any account lookup, creation, or login token is issued, and the ID token's `nonce` claim is checked with `secrets.compare_digest` (constant-time, avoiding a timing side-channel) — both tested directly in `test_oauth.py`.
- **No admin approval gate on self-registration** — accepted, explicitly documented limitation for a course project (`POST /auth/register` accepts any role with no verification). A real identity-verification + admin-approval flow is a backlog item, deliberately not implemented now since retrofitting it would touch registration behavior the test suite, seed script, and every other phase's manual verification currently depend on being immediate.

4.5 Persistent data & complexity

- All persistent state lives in PostgreSQL — no client-trusted state, no file-based storage for anything that matters. Soft-delete (`deleted_at`) is used for patients and users specifically so audit-log rows and ownership relationships (a deleted user can still own historical patients/appointments) are never orphaned by a hard delete.
- The genuinely complex/creative pieces of this system, for grading context: the five-dimension database-enforced conflict-prevention engine and the waitlist's conflict-cause tracking and first-come- first-served re-promotion logic (§3.2–3.3); the deliberately narrow, two-job local-AI trust boundary (§2, §3.7) where the model is never trusted with an ID, a date, or a booking decision; and the Postgres LISTEN/NOTIFY→WebSocket real-time bridge that lets background-worker-originated events (not just same-request events) reach connected clients live.

5. Deployment

- `docker compose up --build` brings up the full stack with one command — migrations, Ollama model pull, and demo-data seeding all run automatically via one-shot init containers, nothing else to run separately. See `README.md` for the full command reference.

6. Known Limitations (honestly stated, not hidden)

- No admin-approval gate on student/attending/admin self-registration (§4.4) — an accepted simplification for a course project, documented as a backlog item, not fixed unprompted.
- Local AI output quality varies with whatever model is actually pulled locally — by design, since the app never trusts the model with a real decision (§2, §4.1).
- Single-instance `db`/`ollama`/`mailhog` — no replica/failover (§4.1); appropriate for a localhost-first course deliverable, explicitly not claimed as production-grade high availability.
- No native mobile app, no real EHR integration, no AI-driven diagnosis/treatment recommendation — all intentional non-goals from the original proposal, not gaps.

7. Curriculum Topic Coverage

How this project maps onto the course's own named units, for a reader checking against the syllabus rather than the proposal:

- **Git**: full real commit history on `github.com/edanperetz2/stu-dent`, feature branches, and a merged pull request.
- **Containers**: the whole stack runs as seven Docker Compose services plus two one-shot init containers (§2, "Backend container topology").
- **Unit tests**: 359 backend + 259 frontend tests (§3).
- **Sockets + IPC**: the real-time layer is a WebSocket endpoint (`/ws`) backed by a Postgres LISTEN/NOTIFY bridge (`app/realtime/listener.py`, §2, §3.9) — a dedicated async task relays DB-side events to connected sockets, including events raised by the `worker` container, not just the `api` process handling the request.
- **Client-server + Python packaging/organization**: a FastAPI backend (`models//schemas//core//servic` one concern per module) serving a fully decoupled React/TypeScript client over HTTP and WebSocket (§2).
- **Parallel programming**: real concurrency handling throughout, not just claimed — Postgres `ExcludeConstraint`s prevent double-booking under concurrent requests (§2), several services use SAVEPOINT-guarded insert-or-update to rescue concurrent duplicate inserts, and `test_stress.py` (§3.10) drives genuinely concurrent load with real threads, not mocks.
- **CI/CD**: a 6-job GitHub Actions pipeline — lint, dependency audit, backend tests, frontend tests, end-to-end tests, and a Docker build check — runs on every push (§3.9).

8. Team Contributions

All three members are real committers on `github.com/edanperetz2/stu-dent` (105 commits in total); the split below is grounded in that history (`git log --author=...`), not a generic "everyone did everything."

- **Idan Peretz** (97 commits) — implemented, across every phase: the repo scaffold and Docker Compose topology; the Phase 1 auth/RBAC layer (argon2, JWT, audit-log-based rate limiting); the scheduling engine with Postgres exclusion constraints and the appointment state machine; waitlists, the background worker, email reminders and notifications; the forum, messaging redesign, and the WebSocket + LISTEN/NOTIFY real-time bridge; the local-AI features (scheduling interpreter, report assistant) and their deterministic trust-boundary design; part of the React frontend (appointments calendar, conflict-resolution/waitlist flow, dark mode, forgot-password, accessibility pass); the CI/CD pipeline, dependency audit, and seed data; the full backend and frontend test suites; and this report and the demo videos; also the two post-submission arcs — the review-driven UX/accessibility remediation and the full-repo audit/fix pass.
- **Yoni Oshin** (7 commits) — originated the project idea and led the frontend planning (page structure, role-aware navigation, and the look the UI was built toward). Implemented, via a merged pull request: the Google OAuth sign-in/registration flow (backend `oauth` routes and service, `oauth_identities` table, frontend `SocialAuthButtons` and callback handoff screen), configurable weekly/monthly report generation, appointment-calendar interaction polish, the logo, favicon, and assorted UI enhancements, plus making the backend test timestamps relative so they don't go stale.
- **Sharbel Makhoul** (1 commit) — reformulated the project proposal (`docs/proposal.md`), the document every phase was scoped and verified against; helped plan and design the project architecture (§2); planned part of the test strategy (§3); and contributed to the visual design and theming of the UI.

9. Summary

Stu-Dent covers the full scope of the original proposal: unified role-based auth, a database-enforced conflict-free scheduling engine across five resource dimensions, a self-healing waitlist, notifications and reminders, a student forum, direct messaging, admin resource management, and a local AI layer for both natural-language scheduling entry and usage reporting — all Dockerized behind one command, with no paid third-party services anywhere in the stack. It's backed by 359 backend tests, 259 frontend tests, and a real end-to-end browser suite, all enforced on every push by CI. The recorded demo video, submitted alongside this report, walks through every workflow described above.